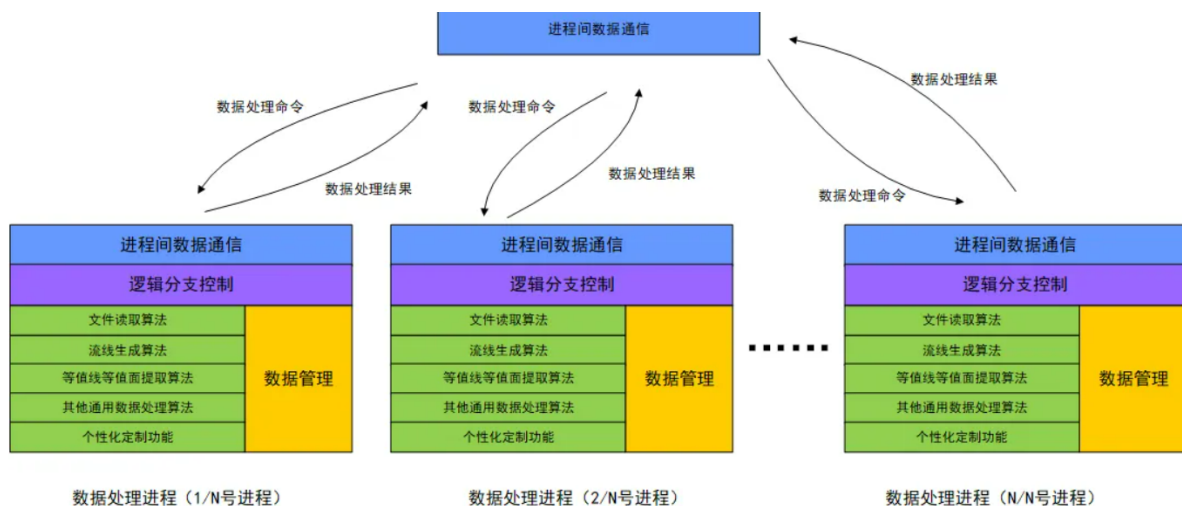


- 1 多进程MPI可以用于并行计算，但是否可以用于一个窗口的渲染（并行渲染）
- 2 阶段结论：
- 3 1）多个进程能否共用一个窗口？
- 4 2）网上说可以，暂时未找到可行方案
- 5 下一步做法：
- 6 1）看fastCAE用法，若可用于渲染，则继续研究；若仅用于计算，则给出阶段性调研报告与结论，停止深入研究。

fastCAE只将多进程用于数据并行处理，而不适用于渲染

参考<https://mp.weixin.qq.com/s/mVwId4YyZ0gKp757qegvpw>



为解决数据处理效率的问题，计划采用MPI多进程并行的方案提升数据处理效率。MPI启动多进程中，0号进程为渲染进程，在该进程中实现数据可视化渲染与用户交互；其他进程均为数据处理进程，数据处理进程彼此独立，中间彼此不进行通信，独立的进行数据读写与数据处理。渲染进程将数据处理命令下发到各个数据处理进程，数据处理进程将输出处理完成之后将结果数据发送回渲染进程进行数据可视化与图形交互操作。

文章中提到0号进程为渲染进程，即只有一个进程用于渲染，故没有用多进程进行渲染，可能是多线程

- 1 2、多核SMP渲染的技术方案
- 2 1）用法：多个方法没有明显差异
- 3 下一步计划：
- 4 1）调研SMP在VTK中用法：VTK官方教程、paraview用法、互联网资料
- 5 2）崩溃处理：
- 6 1）验证SMP用于创建多个actor是否有加速效果（不渲染）
- 7 2）调研添加渲染语句崩溃的原因
- 8 渲染语句只能由UI一个线程调用，其余线程调用会崩溃，原因是其余线程无法处理界面的消息循环。

1.调研SMP在VTK中用法：VTK官方教程、paraview用法、互联网资料

■ paraview源代码中查询SMPTools相关

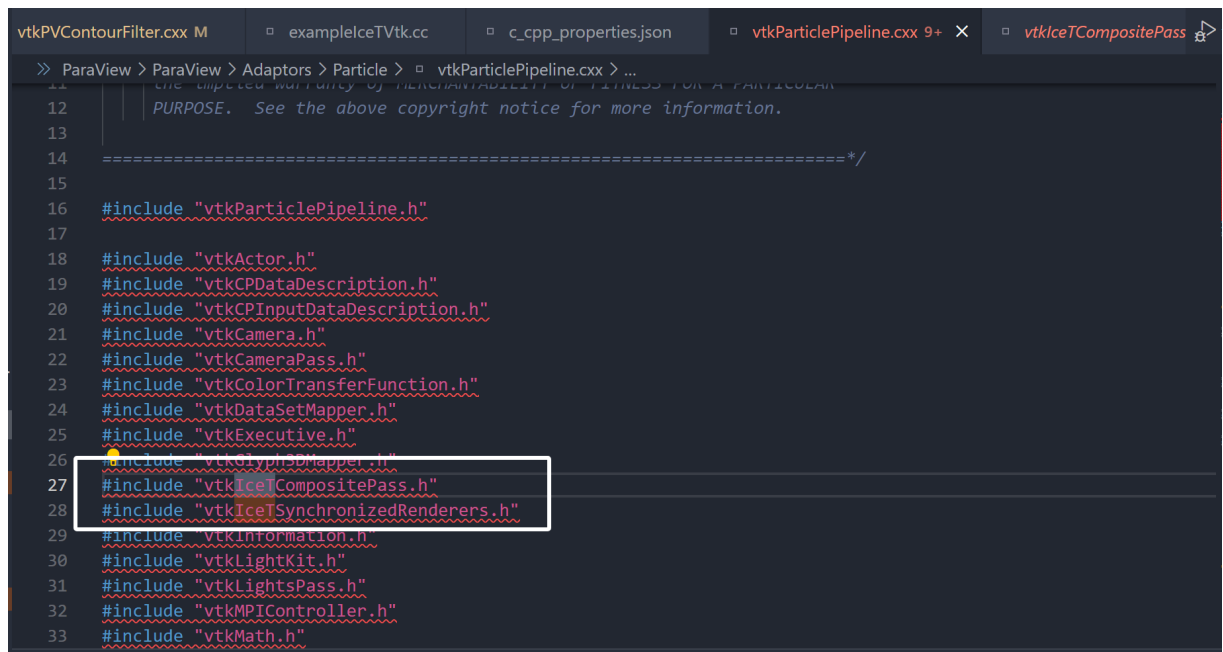
两个SMPTools::For中调用了具体函数的例子：

```
vtkVolumeRepresentation.cxx - 无标题 (工作区) - Visual Studio Code [管理页]
threader.cc  vtkPMaterialClusterExplodeFilter.cxx  vtkParFlowMetaReader.cxx  vtkVolumeRepresentation.cxx 9 X
ParaView > ParaView > Remoting > Views >  vtkVolumeRepresentation.cxx > {} 'anonymous-namespace' > CopyComponent(vtkDataA
{
    vtkSMPTools::For(0, src->GetNumberOfTuples(), [&](vtkIdType start, vtkIdType end) {
        // Compute vector value
        auto inTuples = vtk::DataArrayTupleRange(src, start, end);
        auto outValues = vtk::DataArrayTupleRange(dst, start, end);
        auto outIter = outValues.begin();
        for (auto tuple : inTuples)
        {
            double magnitude = 0.0;
            for (auto component : tuple)
            {
                magnitude += static_cast<double>(component) * static_cast<double>(component);
            }
            (*outIter)[dstComponent] = std::sqrt(magnitude);
            ++outIter;
        }
    });
}

336 {
337     std::set<double> Values;
338
339     template <typename ArrayT>
340     void operator()(ArrayT* scalars) const
341     {
342         VTK_ASSUME(scalars->GetNumberOfComponents() == 1);
343         vtkDataArrayAccessor<ArrayT> accessor(scalars);
344         using ValueT = typename vtkDataArrayAccessor<ArrayT>::APIType;
345
346         vtkSMPTools::For(
347             0, scalars->GetNumberOfTuples(), [this, &accessor](vtkIdType begin, vtkIdType end) {
348                 for (vtkIdType cc = begin; cc < end; ++cc)
349                 {
350                     accessor.Set(
351                         cc, 0, static_cast<ValueT>(this->GetClosest(static_cast<double>(accessor.Get(cc, 0)))));
352                 }
353             });
354     }
355 }
```

结论：Paraview使用SMPTools来进行并行的数据计算工作，没有找到他用来进行并行渲染的例子

- paraview中的并行渲染：<https://www.jishulink.com/post/1920993>
 - IceT库：<https://gitlab.kitware.com/icet/icet>
 - 怎么用？是否能在cpu上运行？多进程还是多线程
 - Paraview中怎样使用IceT
 - 将IceT作为子模块
752-806-068
 - 构造了一些IceT相关的类



```
11  // The copyright notice for this file is as follows:
12  // PURPOSE. See the above copyright notice for more information.
13
14  =====*/
15
16  #include "vtkParticlePipeline.h"
17
18  #include "vtkActor.h"
19  #include "vtkCPDataDescription.h"
20  #include "vtkCPInputDataDescription.h"
21  #include "vtkCamera.h"
22  #include "vtkCameraPass.h"
23  #include "vtkColorTransferFunction.h"
24  #include "vtkDataSetMapper.h"
25  #include "vtkExecutive.h"
26  #include "vtkGlyph3DMapper.h"
27  #include "vtkIceTCompositePass.h"
28  #include "vtkIceTSynchronizedRenderers.h"
29  #include "vtkInformation.h"
30  #include "vtkLightKit.h"
31  #include "vtkLightsPass.h"
32  #include "vtkMPIController.h"
33  #include "vtkMath.h"
```

II.崩溃处理:

1. 验证SMP用于创建多个actor是否有加速效果（不渲染）

使用: 10000:0.789669 100000:346.344

不使用: 10000:0.810583 100000:334.248

结论: 无显著差别

2. 调研添加渲染语句崩溃的原因

渲染语句只能由UI一个线程调用, 其余线程调用会崩溃, 原因是其余线程无法处理界面的消息循环。

icet

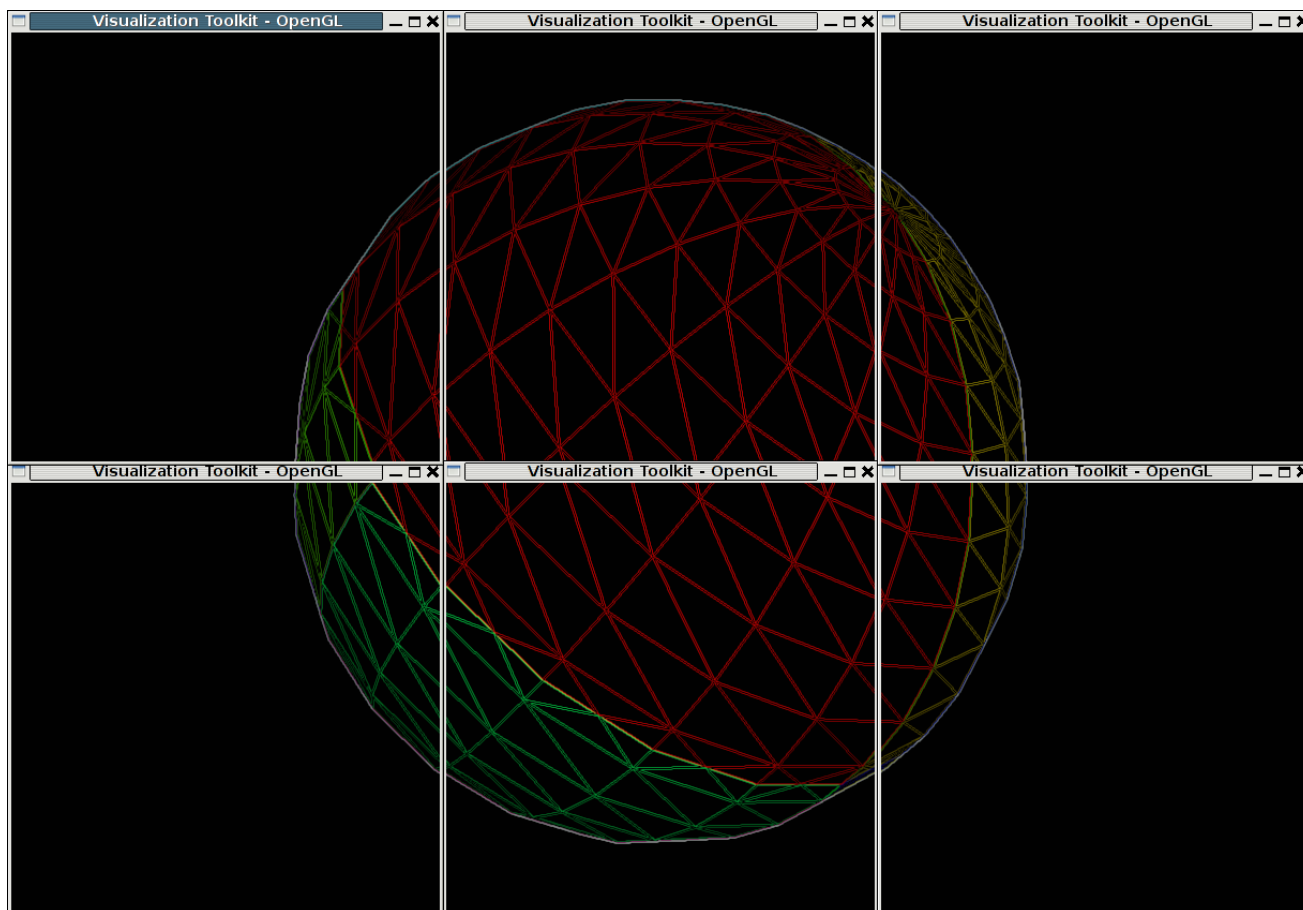
- git clone <https://gitlab.kitware.com/icet/icet.git>

vtk与icet

https://vtk.org/Wiki/VTK/MultiPass_Rendering_With_IceT

自 VTK 5.6 版本以来, 添加了一个多重渲染框架, 以支持需要多次渲染的技术, 例如使用阴影贴图进行阴影渲染。最近, 我们扩展了该框架, 使其能够使用 IceT (用于图块的图像合成引擎) 进行合成。这使得编写在图块显示器或客户端-服务器配置上渲染的并行 VTK 代码成为可能, 同时使用多重渲染框架。

本文档讨论了涉及的各种类以及添加到 VTK/ParaView 存储库中的一些示例实现。



(这个图看起来像是多个框拼接起来的)

vtk or paraview?

VTK 还是 ParaView? 在撰写本文时, IceT 源代码包含在 ParaView 中, 而不是 VTK。因此, 一些特定于 IceT 的类位于 ParaView 存储库中。然而, 我们计划将 IceT 移动到 VTK 中。一旦实现, 这些类也将移动到 VTK 中, 这样编写基于 VTK 的 IceT 示例时就不需要引入 ParaView 了。

所有开发的示例都不依赖任何 ParaView 代码, 除了引入 IceT 库。

开始编写多进程代码前你应该知道的事项

这篇文章仅对编写并行渲染代码的人有兴趣。创建合理的可视化以及所有进程上渲染通道管道的责任完全在于开发者。我们提供了一系列类, 这些类有助于确保所有进程之间的窗口/渲染器/相机同步。但开发者必须使用这些类并正确设置它们

vtkIceTCompositePass

上述示例展示了如何使用 vtkIceTCompositePass。vtkIceTCompositePass 是 vtkRenderPass 的子类, 使用 IceT 进行合成。当在不同进程中实例化时, 它确保合成结果在根节点的活动渲染缓冲区中可用(或在图块显示模式下, 在所有节点上的图块结果可用)。如果不在图块显示模式下, 那么该节点上的活动缓冲区将包含该进程上可用数据的渲染结果, 这可能是部分的, 具体取决于您的管道。

vtkIceTCompositePass 提供了 API, 用于指示是否将设置视为图块显示, 即将所有节点视为形成一个大显示器, 然后仅显示与该进程对应的这个大显示器的图块。

如果数据在所有节点上都被复制, 您可以考虑将 vtkIceTCompositePass::DataReplicatedOnAllProcesses 设置为 true, 以避免不必要的合成。

每当涉及半透明几何体或体绘制时, 需要确保提供一个可以用于生成顺序合成顺序的 KdTree。这可能需要重新分配数据以生成 KdTree。您可能需要查看 vtkDistributedDataFilter 以重新分配数据。

`vtkIceTCompositePass` 像其他任何 `vtkRenderPass` 一样，在活动帧缓冲区上工作。因此，如果您想应用额外的处理，如 Sobel 边缘检测或高斯模糊，可以将 `vtkIceTCompositePass` 添加为 `vtkSobelGradientMagnitudePass` 或 `vtkGaussianBlurPass` 的委托传递。当然，除了在图块显示模式下，所有非根节点将处理部分结果（如果数据是分布的）。

vtkSynchronizedRenderWindows：同步渲染窗口

在处理并行渲染时，需要确保所有渲染窗口具有相同的尺寸。通常情况下，根节点充当驱动程序，在这种情况下，我们需要确保每次根节点渲染时，所有卫星节点也开始渲染。所有这些都由 `vtkSynchronizedRenderWindows` 管理。在过去，使用 `vtkParallelRenderManager` 达到相同效果。然而，`vtkParallelRenderManager` 还同步渲染器，并且一次只支持一个实例。

多个 `vtkSynchronizedRenderWindows` 实例可用于同步多个渲染窗口，但每个实例应分配一个在所有进程中相同的唯一 ID。

vtkSynchronizedRenderers：同步渲染器

`vtkSynchronizedRenderers` 类似于 `vtkSynchronizedRenderWindows`，不同之处在于它同步的是渲染器。每个同步渲染器需要一个独立的 `vtkSynchronizedRenderer`。它不会执行任何合成或其他类似操作。它所做的只是确保所有卫星节点上的渲染器具有与根节点相同的相机参数/视口等。

测试/示例

请参考以下测试，作为如何有效使用这些类的示例：

- `{ParaViewSource}/Servers/Filters/Testing/Cxx/TestSimpleIceTCompositePass.cxx`
该示例在此页面上进行了重现的简单示例。
- `{ParaViewSource}/Servers/Filters/Testing/Cxx/TestIceTCompositePass.cxx`
一个详尽的示例，演示了如何使用 Sobel 边缘检测、模糊、顺序合成、深度剥离等与 `IceTCompositePass` 配合。此示例提供了大量命令行选项（使用 `--help` 显示），可以组合使用以测试各种组件。
- `{ParaViewSource}/Servers/Filters/Testing/Cxx/TestIceTShadowMapPass.cxx`
演示了如何将 `vtkIceTCompositePass` 与复杂渲染通道（如 `ShadowMap` 渲染通道）结合使用。

迁移到 VTK

目前，VTK-ARB 正在讨论对 VTK 目录结构进行更改，以使其更加模块化。作为这一重组的一部分，`IceT` 组件将被迁移到 VTK 中。在此之前，仍需依赖 `ParaView` 来使用 `IceT`。

如何在vtk中使用icet（主要是找不到vtkIceTCompositePass这个类）

- 直接搜索：没有搜到
- vtk论坛上查询：一篇讨论帖，没有使用相关<https://discourse.vtk.org/t/deadline-for-vtk-9-1-rc1-merge-requests-october-29-2020/3206/17>
- 在vtk库中查询

结论：只有在注释中提到paraview里面有派生的类，没有vtk直接的类

```
VContourFilter.cxx M    exampleCpceTVtk.cc    c_cpp_properties.json    exampleCpceTandVTK.cc    vtkRenderer.h X    exampleSMI ▶ ⓘ
```

```
>> vtk-v9.1.0 > include >    vtkRenderer.h > ⓘ vtkRenderer
588   vtkAssemblyPath* PickProp(double selectionX, double selectionY) override
589   {
590       return this->PickProp(selectionX, selectionY, selectionX, selectionY);
591   }
592   vtkAssemblyPath* PickProp(
593       double selectionX1, double selectionY1, double selectionX2, double selectionY2) override;
594   ///@}
595
596   /**
597    * Do anything necessary between rendering the left and right viewpoints
598    * in a stereo render. Doesn't do anything except in the derived
599    * class vtkIceTRenderer in ParaView.
600    */
601   virtual void StereoMidpoint() { return; }
602
603   /**
604    * Compute the aspect ratio of this renderer for the current tile. When
605    * tiled displays are used the aspect ratio of the renderer for a given
606    * tile may be different that the aspect ratio of the renderer when rendered
607    * in it entirety
608    */
609   double GetTiledAspectRatio();
610
```

- paraview库中的例子：还没找到在哪儿